

Sub-module 5

RISC-V AXI Integration

PicoRV32 ↔ AXI-Lite Bridge FSM

For learners with RISC-V ISA and AXI4-Lite fundamentals

What We'll Cover

- | | | | |
|----|---|----|--|
| 01 | Integration Problem Statement | 08 | AXI Write & Read Transactions |
| 02 | PicoRV32 Native Memory Interface | 09 | Design Rules |
| 03 | Why Introduce AXI4-Lite? | 10 | End-to-End: uart_putc('H') Full Signal Trace |
| 04 | THIS Project's Bridge — Custom FSM in top.v | 11 | Firmware Macros — fw/main.c |
| 05 | SoC Architecture Overview | 12 | Lab: make riscv_axi |
| 06 | Bridge FSM: 5 States | 13 | Summary |
| 07 | Bridge Registers & ASIC-Clean Code | | |

Integration Problem Statement

Analogy:

PicoRV32 speaks its own simple handshake language (`mem_valid`/`mem_ready`). AXI4-Lite is a richer, industry-standard protocol used by most SoC IP blocks. To connect them, you need a translator — an adapter — just like a plug adapter when travelling abroad.

PicoRV32 Native Bus

- Uses `mem_valid` to request a transfer
- Waits (stalls) until `mem_ready=1`
- One transaction at a time — very simple
- No channels, no separate handshakes

The Mismatch

- AXI4-Lite has 5 independent channels
- Each channel has its own `VALID/READY`
- Most SoC IP only exposes AXI interfaces
- You cannot wire them together directly

The Solution

- Use `picorv32_axi` wrapper, OR
- Use `picorv32` + `axi_adapter` bridge
- The adapter converts single-channel `mem` bus → 5-channel AXI
- Rest of SoC sees a clean AXI master

Prerequisites assumed:

- ✓ RISC-V ISA — you know how `lw` / `sw` / store instructions work
- ✓ AXI4-Lite — you know the 5 channels (`AW`, `W`, `B`, `AR`, `R`) and `VALID/READY` handshake

PicoRV32 Native Memory Interface

Signal	Dir	Width	Description
mem_valid	OUT	1	CPU asserts: memory request active
mem_ready	IN	1	Slave responds: request accepted
mem_addr	OUT	32	Target byte address
mem_wdata	OUT	32	Write data (byte-lane aligned)
mem_wstrb	OUT	4	Byte enables — non-zero = write, 0 = read
mem_rdata	IN	32	Read data from slave
mem_instr	OUT	1	Instruction fetch (vs data access)

Handshake Rules

1. mem_valid MUST stay HIGH until mem_ready=1
2. mem_wstrb≠0 → Write transaction; =0 → Read transaction
3. One outstanding transaction at a time
4. CATCH_MISALIGN=1 and CATCH_ILLINSN=1 active in this project

Why AXI4-Lite?

① Industry Standard

AXI4-Lite is ARM AMBA spec — used in Xilinx, Intel, SiFive IP.
Familiarity and toolchain support.

② Simple Subset

No bursts, no out-of-order. Single-beat transfers only — matches PicoRV32's one-outstanding-TX constraint perfectly.

③ Peripheral Ecosystem

AXI-Lite slaves (UART, GPIO, SPI) are readily available and reusable. `uart_axi.v` drops in without modification.

④ Verifiable Waveforms

5 named channels produce clear GTKWave traces. `b_state[2:0]` lets students trace every cycle of the handshake.

This Project's Bridge — Custom FSM in top.v

NOT picorv32_axi wrapper — fully visible custom logic

WHY CUSTOM?

- Native bus stays accessible for debug
- Fully visible: rtl/top.v lines 91–224
- Integrates with 3-slave interconnect
- ASIC-clean: no initial blocks, no = value initializers
- b_state[2:0] observable in GTKWave

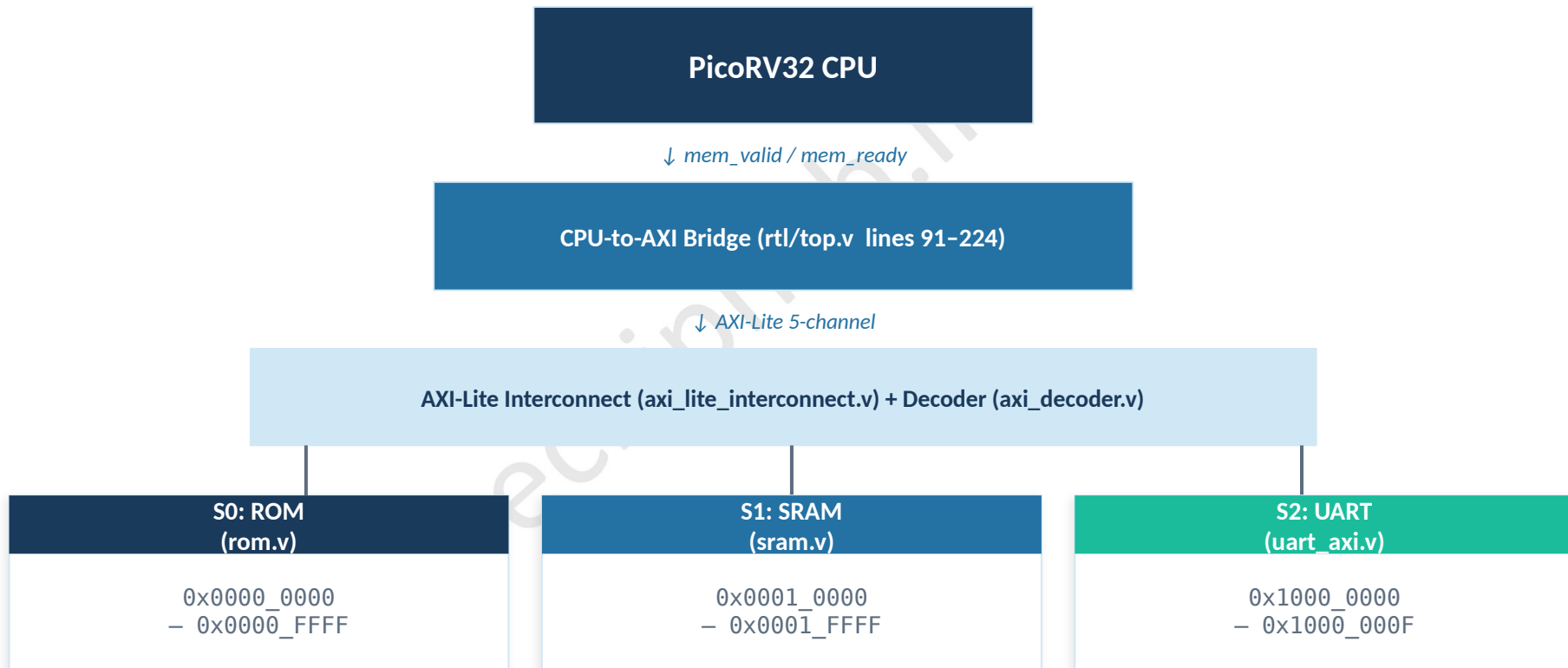
BRIDGE SIGNALS (m_axi_*)

```
// Outputs (CPU → Fabric)
m_axi_awaddr  [31:0]
m_axi_awvalid
m_axi_wdata   [31:0]
m_axi_wstrb   [3:0]
m_axi_wvalid
m_axi_bready
m_axi_araddr  [31:0]
m_axi_arvalid
m_axi_rready

// Inputs (Fabric → CPU)
m_axi_awready
m_axi_wready
m_axi_bvalid
m_axi_arready
m_axi_rvalid
m_axi_rdata   [31:0]
```

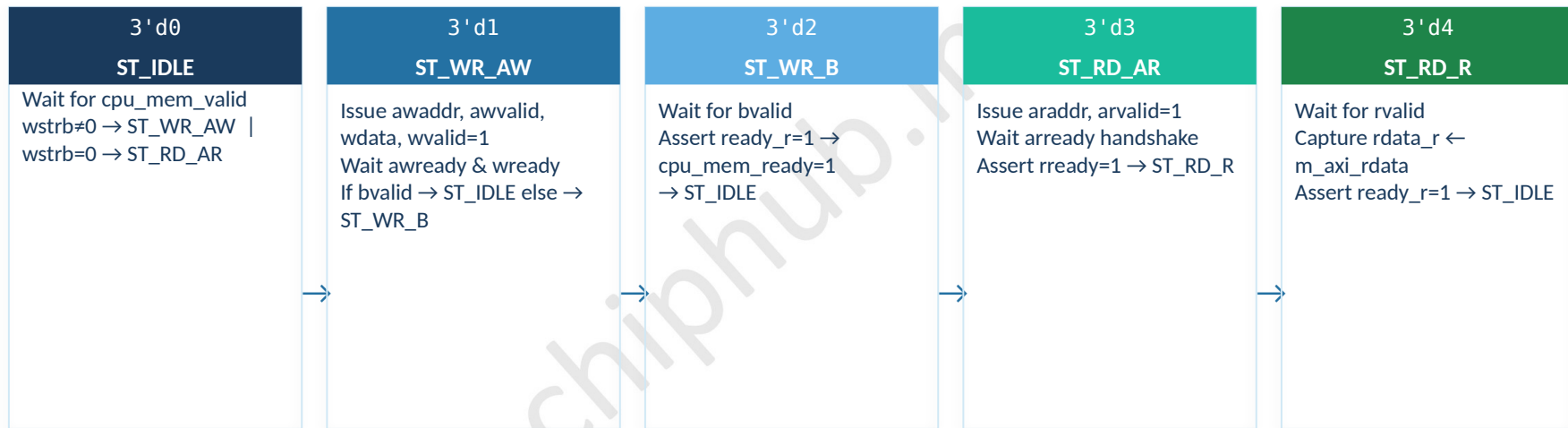
SoC Architecture Overview

How the PicoRV32 master, interconnect, and AXI4-Lite slaves fit together



Bridge FSM: 5 States

b_state[2:0] in rtl/top.v — complete state transitions



Write latency: 3-4 cycles | Read latency: 2-3 cycles | Monitor b_state[2:0] in GTKWave for deadlock detection

Bridge Registers & ASIC-Clean Code

rtl/top.v — all registers reset-driven, no = initializers

```
// Bridge state register
reg [2:0] b_state;

// AXI Write outputs
reg [31:0] m_axi_awaddr;
reg        m_axi_awvalid;
reg [31:0] m_axi_wdata;
reg [3:0]  m_axi_wstrb_r;
reg        m_axi_wvalid;
reg        m_axi_bready;

// AXI Read outputs
reg [31:0] m_axi_araddr;
reg        m_axi_arvalid;
reg        m_axi_rready;

// Internal capture
reg [31:0] rdata_r;
reg        ready_r; // → cpu_mem_ready
```

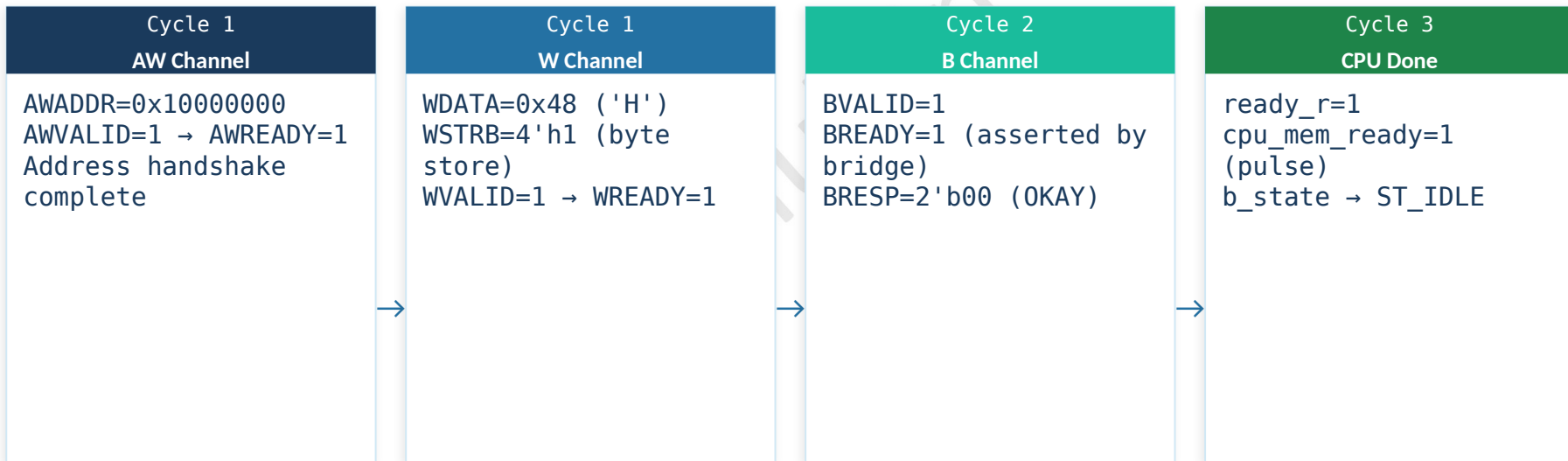
```
// ASIC-clean reset block
if (reset) begin
    b_state      <= ST_IDLE;
    m_axi_awvalid <= 1'b0;
    m_axi_wvalid  <= 1'b0;
    m_axi_bready  <= 1'b0;
    m_axi_arvalid <= 1'b0;
    m_axi_rready  <= 1'b0;
    ready_r       <= 1'b0;
    rdata_r       <= 32'h0;
end

// (* keep *) on all bridge wires
// Prevents Yosys pruning during
// synthesis – critical for ASIC flow

// tx_out renamed from uart_tx
// → avoids Yosys scope collision
```

AXI Write Transaction

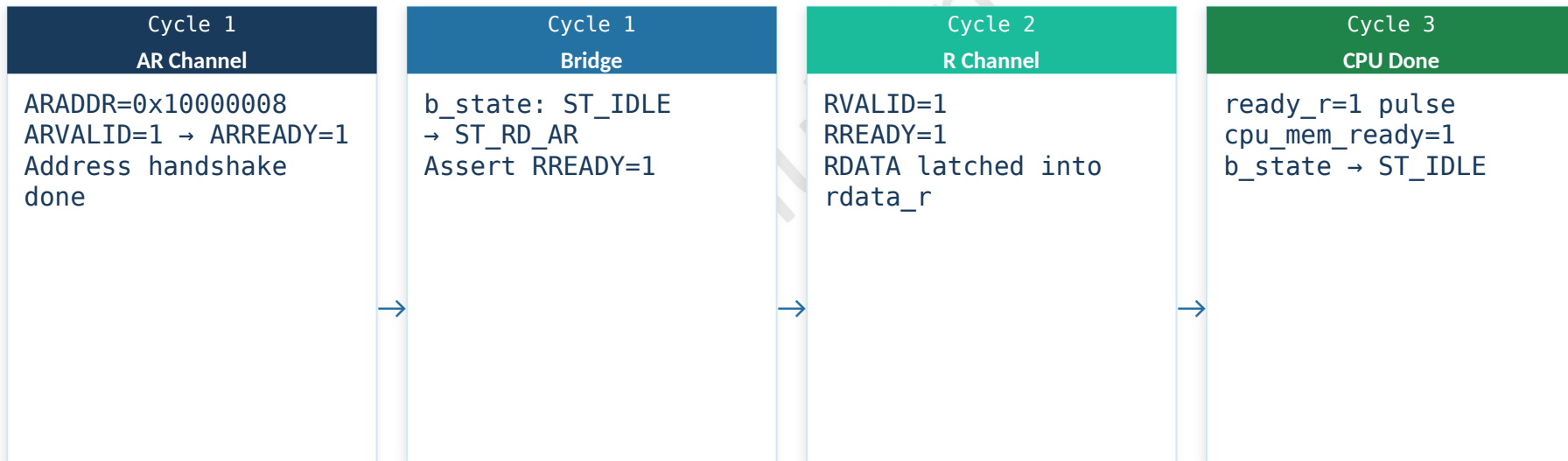
Example: UART TX write → 0x10000000



Note: WSTRB=4'h1 because firmware uses SB (byte store) for UART. AW and W channels can fire simultaneously.

AXI Read Transaction

Example: UART TX read → 0x10000000



Design Rules & Gotchas

KEEP

Rule 1 — VALID Stays High

mem_valid MUST remain asserted until mem_ready=1. Dropping valid early causes the slave to abort — CPU hangs waiting.

KEEP

Rule 2 — One Outstanding TX

Bridge supports one in-flight transaction at a time. b_state must return to ST_IDLE before next request is issued.

NEW

Rule 3 — ASIC-Clean Bridge

All bridge registers use reset-driven initialization. No = value initializers in top.v. Verified Yosys-compatible.

NEW

Rule 4 — Trap on Bad Access

CATCH_MISALIGN=1, CATCH_ILLINSN=1 active. CPU traps if bridge fails to respond — prevents silent hang.

NEW

Rule 5 — tx_out Rename

uart_tx renamed to tx_out port in uart_axi.v. Required to avoid Yosys scope collision during synthesis.

NEW

Rule 6 — Deadlock Detection

If b_state stays non-zero > 10 cycles: check slave valid/ready not toggling. Check wr_sel/rd_sel routing in interconnect.

End-to-End: uart_putc('H') Full Signal Trace

From C firmware to serial bit on tx_out

1. Firmware

```
uart_putc('H')  
→ UART_TX = 'H'  
→ SB x5,0(x10) [wstrb=4'h1,  
addr=0x10000000, wdata=0x48]
```

2. Native Bus

CPU asserts:
mem_valid=1, mem_wstrb=4'h1
mem_addr=0x10000000, mem_wdata=0x48

3. Bridge FSM

ST_IDLE → ST_WR_AW
awaddr=0x10000000, awvalid=1
wdata=0x48, wstrb=4'h1, wvalid=1

4. Interconnect

```
wr_sel=3'b100 (UART selected)  
uart_axi.s_axi_awvalid=1  
Other slaves: awvalid=0
```

5. uart_axi.v

AW handshake → aw_got=1
W handshake → w_byte=0x48,
buf_valid=1
→ BVALID=1, BRESP=2'b00 OKAY

6. uart_tx.v

data=0x48, FSM:
START → DATA(x8) → STOP
tx_out: 8N1 @ 115200 baud

Total latency: ~3 cycle bridge transition + 434 cycle UART start bit (115200 baud @ 50 MHz)

Firmware Macros — fw/main.c

Project address map and UART API

```
#define UART_TX      (*(volatile char*)0x10000000)
#define UART_RX      (*(volatile char*)0x10000004)
#define UART_STATUS (*(volatile int*) 0x10000008)

// No uart_init() needed – uart_axi auto-enabled after reset

static void uart_putc(char c) {
    UART_TX = c;
    // Generates: SB x5, 0(x10)
    // mem_addr=0x10000000, mem_wstrb=4'h1, mem_wdata=c
}

static char uart_getc(void) {
    while (!(UART_STATUS & 0x2)); // poll bit[1] RX ready
    return UART_RX;
}
```

Address	Register	Notes
0x10000000	UART_TX	Byte write → triggers serial TX
0x10000004	UART_RX	Byte read → receive buffer
0x10000008	UART_STATUS	bit[0]=TX busy, bit[1]=RX ready

TX FLOW

```
uart_putc('H')
↓
UART_TX = 0x48
↓
SB x5, 0(x10)
↓
mem_valid=1, wstrb=4'h1
↓
AXI Write → uart_axi.v
↓
tx_out: 8N1 bits
```

Lab — RISC-V AXI Integration Demo

make riscv_axi → AXI handshake walk-through

```
$ make riscv_axi
```

```
---- Simulation Start ----  
[50 ns] AXI WRITE TRANSACTION  
  AWADDR=0x10000000, AWVALID=1 → AWREADY=1 → Address accepted  
  WDATA=0x000000A5, WVALID=1 → WREADY=1 → Data accepted  
  BVALID=1 → Write OKAY → Transaction complete  
  Result: 0xA5 written to 0x10000000  
[110 ns] AXI READ TRANSACTION  
  ARADDR=0x10000000, ARVALID=1 → ARREADY=1 → Address accepted  
  RDATA=0x000000A5, RVALID=1 → RREADY=1 → Data received  
  Result: Read 0xA5 (matches written value)  
Write = Success | Read = Success  
---- Simulation End ----
```

GTKWave Signals

clk, reset

m_awvalid, s_awready
← AW channel

m_wdata, m_wvalid, s_wready
← W channel

s_bvalid, m_bready
← B channel

m_arvalid, s_arready
← AR channel

s_rdata[31:0], s_rvalid
← R channel

b_state[2:0] from tb_top.fst
← Full SoC bridge debug

Key Takeaways



Bridge translates mem_valid/mem_ready → 5-channel AXI-Lite. Implemented in rtl/top.v lines 91-224.



Write path: ST_IDLE → ST_WR_AW → ST_WR_B → ST_IDLE | Read path: ST_IDLE → ST_RD_AR → ST_RD_R → ST_IDLE



All bridge registers: m_axi_* — ASIC-clean, reset-driven, no = initializers. b_state[2:0] is the FSM register.



Project addresses: ROM=0x0000_0000 | SRAM=0x0001_0000 | UART TX=0x1000_0000 | RX=0x1000_0004 | STATUS=0x1000_0008



Lab: make riscv_axi → observe step-by-step AXI handshake log. Testbench: tb/tb_riscv_axi_lite.v



GTKWave: add b_state[2:0], m_axi_awvalid, m_axi_rvalid from tb_top.fst to trace full bridge activity.